

Word Search II Notes

Asked in: Cisco, Snowflake

Brute Force Approach

Bruteforce approach to check word by word if it exists in board just like word search 1.

```
class Solution {
public:

    vector<pair<int,int>> dir={{-1,0},{0,-1},{0,1},{1,0}};
    int n;
    int m;
    bool dfs(vector<vector<char>>& board, string word,int r , int c,int idx){

        if(idx==word.size()){
            return true; //all char match
        }

        if(r<0 || c<0 || r>=n || c>=m) return false;

        if(word[idx]!=board[r][c]){
            return false;
        }

        //base SEB rule in dfs backtrack questions
        //base cases, seen , explore , backtrack

        char temp=board[r][c]; //saving state of board for backtrack
        board[r][c]='?';

        for(auto &it:dir){ //exploring
            int nr=r+it.first;
            int nc=c+it.second;

            if(dfs(board,word,nr,nc,idx+1)){
                board[r][c]=temp; //restore before return
            }
        }
    }
};
```

```

        return true;
    }
}

//backtracking
board[r][c]=temp;
return false;
}

bool exist(vector<vector<char>>& board, string word) {
    n=board.size();
    m=board[0].size();

    //har ek cell ko as starting point of dfs choose krenge
    //kyunki maan lo chahiye tha aadya , and start hi d se kiya to dya mil
    jaayega but what bout start 2 a

    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            if(dfs(board,word,i,j,0)){ //matching starts from 0th index of word
                return true;
            }
        }
    }

    return false;
}

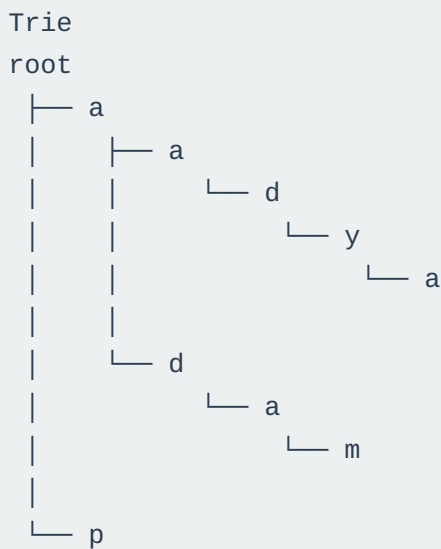
vector<string> findWords(vector<vector<char>>& board, vector<string>& words) {
    vector<string> ans;
    for(string s:words){
        if(exist(board,s)){
            ans.push_back(s);
        }
    }
    return ans;
}

};

```

Approach 2 (Optimal) - Build Trie

Insert every word. Example: **aadya**, **adam**, **apple**



Now DFS. At every cell, suppose **Aadya** has built:

AAD

Instead of asking **Does word[3]=='Y'?** we ask **Does Trie have child Y?**

If No → Immediately stop.

Standard DFS + Backtracking Template

State: Current row, Current column, Current Trie node

Choose: Current character

Explore: 4 directions

Backtrack: Restore board

Word Search I vs Word Search II

Word Search I

Suppose Aadya asks: *"Can you find APPLE?"*

You know exactly one word. So your DFS state is $(r, c, index)$ because you're matching:

A → P → P → L → E

You always know the next expected character.

Word Search II

Now Aadya says: APPLE, APP, APE, BALL, BAT, BAD, CAT, DOG...

30,000 words. What should your DFS state be?

You don't know. After matching A → P, which word are you matching? APP? APPLE? APE? You don't know.

Instead of storing APP, APPLE, APE individually, it stores:

```
A → P → P → L → E
          \
           E
```

Notice **APP** is explored only once.

Suppose Aadya has currently formed **AP**. Now she asks: *"Does any word begin with AP?"*

Trie answers immediately. If No → Stop. No need to explore thousands of useless paths.

Without Trie: Suppose words are APPLE, APP, APE, APRIL, APPLICATION, APEX. After matching AP you would still have to check APPLE?, APP?, APE?, APRIL?, APPLICATION?, APEX? one by one.

With Trie: After matching AP you simply move: `temp = temp->child['P'];`

Now the Trie node itself already represents *All words beginning with AP*. Now I'll attach code, personalize the notes according to my code writing style and approach.

Optimal Code Implementation

```
class TrieNode{
public:

    unordered_map<char,TrieNode*> child; //not actually parent child
                                         //ch->pointer to child nodes
                                         //ex "aa" -> "aad" , "aaz" etc

    bool end;
    char val;
    string word;

    //these are basic building blocks of a trienode
```

```

TrieNode(){
    child.clear();
    end=false;
    word="";
}

TrieNode(char x){
    val=x;
    end=false;
    word="";
}

};

class Trie{
public:

    TrieNode* root;
    Trie(){
        root=new TrieNode();
    }

    void insert(string &s){
        TrieNode* temp=root;
        for(char c:s){
            //agar path already exist nahi kr rha to new node bnana hoga
            if(temp->child.find(c)==temp->child.end()){
                temp->child[c]=new TrieNode(c);
            }

            //agar he already to keep moving
            temp=temp->child[c];
        }

        temp->end=true;
        temp->word=s;
    }
};

int n;
int m;

vector<pair<int,int>> dir={{-1,0},{0,-1},{1,0},{0,1}};

```

```

void dfs(vector<vector<char>>& board,int r,int c,TrieNode* temp,vector<string>& ans){

    if(r<0 || c<0 || r>=n || c>=m) return;
    if(board[r][c]=='?') return;

    char currch=board[r][c];

    //if no trie branch is matching with addition of this char to temp , simply
    return as no match
    if(temp->child.find(currch)==temp->child.end()) return;

    temp=temp->child[currch]; //deep penetration
    if(temp->end){ //if this represent a complete word push to ans
        ans.push_back(temp->word);
        temp->end=false; //to avoid duplicate st never visited again
    }

    board[r][c]='?'; //SEB rule starts , seen,explore,backtrack
    for(auto &it:dir){
        int nr=r+it.first;
        int nc=c+it.second;
        dfs(board,nr,nc,temp,ans);
    }

    board[r][c]=currch;
}

class Solution {
public:

    Trie t;
    vector<string> findWords(vector<vector<char>>& board, vector<string>& words) {
        n=board.size();
        m=board[0].size();

        for(string &s:words){
            t.insert(s);
        }

        vector<string> ans;

        for(int i=0;i<n;i++){
            for(int j=0;j<m;j++){
                dfs(board,i,j,t.root,ans);
            }
        }
    }
};

```

```
        }  
    }  
  
    return ans;  
}  
};
```